

SCX 架构清单：基底、脚本、接口、数学公式、应用场景

日期：2026-06-27 | SCX v0.4.0-pre

0. 回答你的核心问题

Q: 做势函数时用 ACE，做 LLM/图像时用别的神经网络——怎么控制？

A: 通过 **Encoder** 抽象层控制。SCX 不关心你的神经网络是什么结构，只要求你实现一个 `SCXStateEncoder` 接口（3 个方法：`encode`, `distance`, `cluster`）。MLIP 用 `MLIPEncoder`（手工描述符），图像用 `VisionEncoder`（CNN backbone），表格用 `TabularEncoder`。LLM 你写一个 `LLMEncoder` 把文本 embedding 成向量就行。

Q: 基底和应用场景的脚本由接口链接？基底一样吗？

A: 基底完全一样。`SCXStateEncoder` 和 `SCXExpert` 是两个抽象基类，是唯一的领域接口。Encoder 以上的一切（`StateDiscovery`、`DataClassifier`、`StateValue`、`ActionPolicy`）不感知你是什么领域——它们只看到向量。换领域 = 换 Encoder 实现 + 换 YAML 配置。

1. 总体架构图

SCX v4.0 通用模块化架构

应用场景层 (domains/*.yaml)

mlip.yaml cifar.yaml health.yaml (你的新域)

接口层

SCXStateEncoder (抽象基类)

SCXExpert (抽象基类)

基底

核心引擎 (不感知领域)

StateDiscovery → DataClassifier → StateValue

NoiseScore → RedundancyScore → LearnabilityScore

ExpertReliability → ExpertRouter → CompressStrategy

ActionPolicy → SCXFramework (orchestrator)

关键设计原则：Encoder 以上不变，Encoder 以下是领域适配。新领域只需实现 Encoder + 写 YAML。

2. 基底（核心引擎）——所有应用共享，不随领域变化

2.1 基底的 Python 脚本清单

文件	类/函数	作用	依赖领域吗？
core/config.py	SCXConfig	全局配置（状态数、阈值、预算等），从 JSON/YAML 加载	
core/framework.py	SCXFramework	总调度器：fit() → 状态发现 → 估值 → 分类 → 动作	
core/metrics.py	SCXMetrics	指标记录与追踪	
core/online.py	OnlineStateTracker, OnlineExpertTracker, OnlineSCXFramework	流式/在线场景的 EMA 更新	
state/discovery.py	StateDiscovery	聚类状态发现 (KMeans/GMM/Spectral/HDBSCAN)	
state/space.py	StateSpace	状态空间数据结构	
state/assignment.py	StateAssignment	新样本映射到已有状态	
state/metrics.py	状态距离/相似度度量	状态间距离计算	

文件	类/函数	作用	依赖领域吗？
state/robustness	StateRobustness	状态鲁棒性分析 (misclassification/boundary confidence)	
state/two_layer	TwoLayerStateDiscovery	两层描述符： L1 人工 + L2 误差驱动	
valuation/classifier	DataClassifier	四分类器 (valuable/redundant/noisy/expert_dep)	
valuation/state	StateValue + theorem-based methods	V(s) 计算 [deprecated], noise_consistency_score, cher- noff_bound, hoeffd- ing_bound, fea- ture_strength_diagnostic	
valuation/noise	NoiseScore	样本级/状态级 噪声评分	
valuation/redundancy	RedundancyScore	冗余度评分	
valuation/learnability	LearnabilityScore	可学习性评分 (一致性 + 非 噪声)	

文件	类/函数	作用	依赖领域吗？
valuation/adapt.py	AdaptiveThreshold	自适应阈值校准（网格搜索/百分位/gap法）	
valuation/influence.py	StateConditionedInfluence	状态条件影响力函数	
yajie.py	YajieCleaner	雅洁数据清理器（定理驱动去噪）	
expert/registry.py	ExpertRegistry	专家注册与管理	
expert/reliability.py	ExpertReliability	状态条件专家可靠性 SCX_m(s)	
expert/router.py	ExpertRouter	专家路由 (hard/soft)	
expert/conflict.py	专家冲突解决	多专家冲突检测与仲裁	
action/compress.py	CompressStrategy	数据压缩策略	
action/acquisition.py	主动采集策略	状态条件采样	
action/policy.py	ActionPolicy	动作决策策略	
utils/data_loader.py	数据加载工具	统一数据加载接口	
utils/evaluation.py	评估工具	指标计算、对比	
utils/visualization.py	可视化工具	状态地图、残差图、对比图	

文件	类/函数	作用	依赖领域吗？
<code>utils/helpers.py</code>	通用辅助函数	数组/统计工具	

总计：31 个源文件，全部领域无关。

2.2 基底使用的数学公式

核心定义

公式	含义	代码位置
$R_m(s) = E[(f_m(x), f^*(x)) x s]$	专家 m 在状态 s 的条件风险	<code>expert/reliability.py</code>
$SCX_m(s) = P((f_m(x), y) < x s)$	专家 m 在状态 s 下做出可接受预测的概率	<code>expert/reliability.py</code>
$V_add(s) = \bar{r}(s) \cdot (s) \cdot L(s) \cdot [1-D(s)] \cdot \max_m SCX_m(s)$	采集价值（越采集越值钱）	<code>valuation/state_value.py</code>
$V_remove(s) = (s) \cdot (1-\bar{r}(s)) \cdot Sim(s) \cdot (1-Boundary(s))$	压缩价值（越压缩越安全）	<code>valuation/state_value.py</code>
$NoiseScore(x) = r \cdot w_ /((s)+) \cdot [1-C(s)] \cdot w_C$	噪声评分（高误差 + 低密度 + 低一致性 → 噪声）	<code>valuation/noise_score.py</code>

公式	含义	代码位置
$L(s) = C(s) \cdot [1 - N(s)]$	可学习性 = 一致性 $\times (1 - \text{噪声})$	valuation/learnability.py
$D(s) = f(\cdot, \bar{r}, \text{Similarity}, \text{Boundary})$	冗余度	valuation/redundancy.py

数据四分类规则

IF expert_gap > threshold → expert_dependent (路由)
 IF $\bar{r} \uparrow$ AND $\uparrow$ AND $C \uparrow$ AND $D \downarrow$ → valuable (采集)
 IF $\bar{r} \downarrow$ AND $\uparrow$ AND $D \uparrow$ → redundant (压缩)
 IF $\bar{r} \uparrow$ AND $\downarrow$ AND $C \downarrow$ → noisy (降权/丢弃)
 ELSE → valuable (保守)

代码: valuation/classifier.py 的 DataClassifier.classify_state()

两层描述符 (Proposition 6)

Layer 1: $_human(x) \rightarrow KMeans \rightarrow \text{coarse states } \{S_1, S_2, \dots, S_K\}$

Layer 2: S_k :

1. 计算 $MI(\text{feature_dim}, \text{error})$ for each feature dim
2. 选 Top-d 误差相关维度
3. 仅在误差子空间聚类 → error_states
4. 输出两层级联划分

代码: state/two_layer.py 的 TwoLayerStateDiscovery.discover()

3. 接口层（Encoder 抽象基类）——连接基底与应用

3.1 SCXStateEncoder（核心接口）

```
class SCXStateEncoder(ABC):
    @abstractmethod
    def encode(self, x: Any) -> np.ndarray:
        """ $x \rightarrow (x) \quad R^d$ """

    @abstractmethod
    def distance(self, a: np.ndarray, b: np.ndarray) -> float:
        """ 特征空间中的距离 """

    @abstractmethod
    def cluster(self, X: np.ndarray, n_clusters: int, **kw) -> tuple[labels, centroids]:
        """ 聚类划分状态 """

    def batch_encode(self, X: list) -> np.ndarray:
        """ 批量编码（可重写做并行） """

    def get_default_config(self) -> dict:
        """ 默认状态发现配置 {n_states, cluster_method} """

    def get_feature_dim(self) -> int:
        """ 返回特征维度  $d$  """
```

这就是唯一必须实现的接口。无论你做 MLIP、CNN、Transformer、LLM embedding——只要实现这 3 个抽象方法，SCX 的整个基底就能跑。

3.2 SCXExpert（轻量专家接口）

```
class SCXExpert(ABC):
    @abstractmethod
    def predict(self, x: Any) -> Any:
        """ 专家预测 """

    def cost(self) -> float:
        """ 调用成本 (默认 1.0) """
```

4. 领域适配层（Encoder 实现 + YAML 配置）

4.1 已实现的 Encoder

Encoder	文件	用于什么网络/模型	特征维度	领域
MLIPEncoder	encoders/mlip.py	ACF/NEP/MACE 势函数（手工描述符：配位数/键长/体积...）	12-dim	材料/DFT
VisionEncoder	encoders/simple_cnn.py	SimpleCNN / ResNet18 / ResNet50	256-dim (simple_cnn)	图像分类
TabularEncoder	encoders/tabular.py	任何表格模型（归一化 + one-hot）	可变	表格数据
ErrorDrivenEncoder	encoders/layer_2.py	Layer 2 专用，包装任意 Layer 1 encoder	可变（误差相关子空间）	全部

4.2 如何为新领域写 Encoder (3 步)

例：为 LLM 文本分类写 Encoder

```
from scx.encoders.base import SCXStateEncoder
import numpy as np
```

```
class LLMEncoder(SCXStateEncoder):
```

```
    def __init__(self, model_name="bert-base"):
```

```
        self.model = load_embedding_model(model_name) # 你的 embedding 模型
```

```
        self._feature_dim = 768
```

```
    def encode(self, text: str) -> np.ndarray:
```

```
        return self.model.embed(text) # text -> 768-dim vector
```

```
    def distance(self, a, b):
```

```
        return float(np.linalg.norm(a - b)) # 欧氏距离
```

```
    def cluster(self, X, n_clusters=10, **kw):
```

```
        from sklearn.cluster import KMeans
```

```
        km = KMeans(n_clusters=n_clusters, random_state=42)
```

```
        return km.fit_predict(X), km.cluster_centers_
```

3 步接入新领域：1. 实现 SCXStateEncoder (如上) 2. 写 YAML 配置 (domains/llm.yaml) 3. `DomainRegistry.register("llm", "domains/llm.yaml")`

4.3 已注册的领域配置

领域	YAML 文件	Encoder	专家	阈值特色
mlip	domains/mlip.yaml	MLIPEncoder (12-dim ACE 描述符)	ACE, NEP, MACE, DFT	error_high=0.05 eV/atom
cifar_image	domains/cifar.yaml	visionEncoder (simple_cnn)	cnn_small, cnn_large, resnet50	error_high=0.3 (loss)
health_image	domains/health.yaml	visionEncoder (simple_cnn, 224x224)	ai_v1, ai_v2, ju- nior_doctor, se- nior_doctor	expert_gap=0.25 (人机混合)

5. 完整脚本作用清单

5.1 基底脚本（src/scx/）

src/scx/	
__init__.py	导出 SCXFramework, SCXConfig, 所有 Encoder, Doma
core/	
config.py	SCXConfig — 全局配置 (K, thresholds, budget)
framework.py	SCXFramework — 总调度器 fit/compress/value/save
metrics.py	SCXMetrics — 指标追踪
online.py	OnlineSCXFramework — 流式/在线版本
encoders/	

base.py	SCXStateEncoder, SCXExpert — 抽象基类（唯一接口）
mlip.py	MLIPEncoder — ACE/势函数（12-dim 手工描述符）
vision.py	VisionEncoder — 图像（CNN/ResNet backbone）
tabular.py	TabularEncoder — 表格数据
error_driven.py	ErrorDrivenEncoder — Layer 2 错误驱动聚类
domains/	
registry.py	DomainRegistry — 注册/查找/加载 YAML 配置
mlip.yaml	MLIP 领域配置
cifar.yaml	CIFAR 领域配置
health.yaml	Medical 领域配置
state/	
discovery.py	StateDiscovery — 聚类
two_layer.py	TwoLayerStateDiscovery — 两层描述符管线
assignment.py	StateAssignment — 新样本映射
space.py	StateSpace — 状态空间数据结构
metrics.py	状态间距离/相似度
robustness.py	StateRobustness — 鲁棒性分析
valuation/	
classifier.py	DataClassifier — 四分类器
state_value.py	StateValue — $V(s)$ [deprecated] + theorem-based
noise_score.py	NoiseScore — 噪声评分
redundancy.py	RedundancyScore — 冗余评分
learnability.py	LearnabilityScore — 可学习性
adaptive.py	AdaptiveThreshold — 自适应阈值
influence.py	StateConditionedInfluence — 影响力
yajie.py	YajieCleaner — 雅洁数据清理器（定理驱动去噪）
expert/	
registry.py	ExpertRegistry — 专家注册
reliability.py	ExpertReliability — $SCX_m(s)$ 计算

router.py	ExpertRouter — hard/soft 路由
conflict.py	专家冲突仲裁
action/	
compress.py	CompressStrategy — 压缩
acquisition.py	主动采集
policy.py	ActionPolicy — 决策策略
utils/	
data_loader.py	数据加载
evaluation.py	评估工具
visualization.py	可视化
helpers.py	辅助函数

5.2 实验脚本 (experiments/)

脚本	作用	调用的 SCX 模块
mlip_case/run_scx_on_ain_v3.py	一层 SCX 分析 AIN v3 (534 帧)	MLIPEncoder, StateDiscovery, DataClassifier, NoiseScore, StateValue
mlip_case/run_scx_on_ain_v3_two_layer.py	两层 SCX 分析 AIN v3	MLIPEncoder + ErrorDrivenEncoder + TwoLayerStateDiscovery
mlip_case/compare_ain_files.py	SCX 分析结果对比	读取 v3 训练结果 + SCX 结果
mlip_case/analyze_overlap.py	噪声帧重叠分析	交叉对比 SCX 噪声标记 vs REPORT 最差帧
mlip_case/final_check.py	最终验证统计	Pearson r, RMSE 预估值
cifar/run_cifar100_noise.py	CIFAR-100 噪声检测实验	VisionEncoder, NoiseScore

脚本	作用	调用的 SCX 模块
cifar/run_cifar100_compress.py	CFAR100 压缩实验	VisionEncoder, CompressStrategy
cifar/run_cifar100_routing.py	CFAR100 路由实验	VisionEncoder, ExpertRouter
cifar/run_baseline.py	baseline 对比	DataClassifier vs Random/LOO/Shapley
synthetic/data_generator.py	合成数据生成	生成 2D 状态条件数据集
synthetic/run_experiment.py	合成数据验证	全管线
(待实现)	蒸馏去病毒化	Teacher 势函数蒸馏数据 + SCX 两层过滤
distill/devirus.py		

5.3 scx-health 实验脚本 (scx-health/experiments/)

脚本	作用	数据集
compress/run_compress.py	SCX-Compress 完整版 (含 Coreset)	PathMNIST 90K
compress/run_compress_v2_quick.py	SCX-Compress 快速版 (无 Coreset)	PathMNIST 90K
noise/run_noise.py	SCX-Noise 均匀 + 按类别噪声	DermaMNIST 7K
routing/run_routing.py	SCX-Routing 多专家路由	BloodMNIST 12K

5.4 测试脚本 (tests/)

文件	覆盖模块	测试数
test_state.py	StateDiscovery, StateSpace, StateAssignment, 两层	~100
test_valuation.py	DataClassifier, NoiseScore, StateValue, Learnability, Redundancy	~80
test_expert.py	ExpertRouter, ExpertReliability	~60
test_expert_module.py	ExpertRegistry	~30
test_action.py	CompressStrategy, ActionPolicy	~50
test_two_layer.py	TwoLayerStateDiscovery, ErrorDrivenEncoder	~30
test_robustness.py	StateRobustness	~20
test_online.py	OnlineSCXFramework	~20
test_utils.py	辅助函数	~10
test_yajie.py	YajieCleaner	~3
conftest.py	共享 fixtures	—

总计 ~427 tests

6. 不同应用场景的调用方式

6.1 MLIP 场景 (AIN v3)

```
from scx.encoders.mlip import MLIPEncoder
from scx.state.discovery import StateDiscovery
from scx.valuation.classifier import DataClassifier
from scx.valuation.noise_score import NoiseScore

# 1. 编码
encoder = MLIPEncoder(species=["Al", "N"])
features = encoder.batch_encode(atoms_list) # 534 x 12

# 2. 状态发现
discovery = StateDiscovery(method="kmeans", n_states=20)
labels = discovery.fit_predict(features)

# 3. 计算残差 (fmax)
fmax = compute_fmax(atoms_list)

# 4. 四分类
classifier = DataClassifier({"error_high": 3.0}) # MLIP 专用阈值
for s in states:
    cat = classifier.classify_state(r_bar, prop, consistency, redundancy, noise)

# 5. 两层方法
from scx.state.two_layer import TwoLayerStateDiscovery
two_layer = TwoLayerStateDiscovery(encoder)
result = two_layer.discover(atoms_list, fmax, layer1_k=20, layer2_k=5)
```

6.2 图像场景（CIFAR/MedMNIST）

```
from scx.encoders.vision import VisionEncoder

# 编码器换成 VisionEncoder, 其他一样
encoder = VisionEncoder(backbone="simple_cnn")
features = encoder.batch_encode(images) #  $N \times 256$ 
# 后续管线完全不变
```

6.3 表格场景

```
from scx.encoders.tabular import TabularEncoder

encoder = TabularEncoder(normalize=True)
features = encoder.batch_encode(dataframe) #  $N \times d$ 
# 后续管线完全不变
```

6.4 LLM 场景（待实现）

```
# 只需写一个 LLMEncoder, 其他全不变
from scx.encoders.base import SCXStateEncoder

class LLMEncoder(SCXStateEncoder):
    def encode(self, text): return embedding_model(text)
    def distance(self, a, b): return cosine_distance(a, b)
    def cluster(self, X, n, **kw): return KMeans(n).fit_predict(X), centroids

# 后续管线完全不变
```

7. 数据流图（一次完整的 SCX 调用）

输入数据 X （原子构型/图像/文本/表格行）

`Encoder.encode()` $\leftarrow$ 领域唯一差异点

$X \rightarrow (X) \quad \mathbb{R}^d$

$(X) \quad \text{numpy array } (N, d)$

`StateDiscovery`

`cluster( (X), K)`

$\rightarrow \text{labels, centroids}$

$\text{state_labels } (N,)$

残差计算

$r_i = (f(x_i), y_i) \leftarrow \text{需要模型预测+标签}$

$\text{residuals } (N,)$

四维估值 (per state)

$\bar{r}(s)$ (s) L(s) D(s)

NoiseScore(s)

SCX_m(s) for each m

state_metrics dict

DataClassifier

classify_state()

→ Valuable/Redun/

Noisy/ExpertDep

category per state

ActionPolicy

→ acquire/compress

/downweight/route

反馈（闭环）

真实结果回传

重校准阈值

第 步是唯一因领域而异的部分。 - 对所有领域完全相同。

8. 两类两层的实现对照

	EGP 两层 (V3 Phase C)	SCX 两层 (v4.0)
目的	合并势函数时消除参数冗余	发现噪声/冗余/高价值状态
Layer 1	shared c_0 (共用)	人工描述符粗聚类
Layer 2	element correction c_Z	ErrorDrivenEncoder (误差子空间聚类)
操作对象	模型系数	训练数据
关键操作	post-hoc projection (violation→0)	互信息筛选 + 误差子空间聚类
实物产出	gauge-fixed 势函数	数据四分类 + 去噪建议

9. 关键对照表：什么场景用什么

你要做的事	用的 Encoder	用的 YAML	用的关键基底模块
分析 AIN DFT 训练数据质量	MLIPEncoder	mlip.yaml	TwoLayerDiscovery, DataClassifier, NoiseScore
CIFAR 图像去噪/压缩	VisionEncoder (simple_cnn)	cifar.yaml	NoiseScore, CompressStrategy
MedMNIST 医学图像路由	VisionEncoder (simple_cnn)	health.yaml	ExpertRouter, DataClassifier

你要做的事	用的 Encoder	用的 YAML	用的关键基底模块
表格数据估值 (UCI)	TabularEncoder	(新建 tabular.yaml)	StateValue, DataClassifier
LLM embedding 分 析	(待写 LLMEncoder)	(新建 llm.yaml)	全部通用
FEM 仿真参 数空间	(待写 FEMEncoder)	(新建 fem.yaml)	TwoLayerDiscovery, StateValue
在线/流式数 据	任意 Encoder	任意 YAML	OnlineSCXFramework, OnlineStateTracker

10. 一句话总结

Encoder 是唯一的领域插头，基底是不变的核心引擎。
MLIP 用 MLIPEncoder（手工描述符），图像用 VisionEncoder（CNN），LLM 你写个 LLMEncoder。
插头一换，整个 SCX 管线就能在新领域跑起来——
StateDiscovery、DataClassifier、NoiseScore、ExpertRouter 全部复用。